

1. Описание условия задания

Цель работы: получить навыки работы с двоичными деревьями поиска. Построить двоичное дерево поиска из букв вводимой строки. Вывести его на экран в виде дерева. Выделить цветом все буквы, встречающиеся более одного раза. Удалить из дерева эти буквы. Вывести оставшиеся элементы дерева при постфиксном его обходе. Сравнить время удаления повторяющихся букв из дерева и из строки. Реализовать основные операции работы с деревом: обход дерева, включение, исключение и поиск узлов. Сравнить эффективность алгоритмов сортировки и поиска значения в зависимости от высоты деревьев и степени их ветвления (при равном количестве узлов).

2. Техническое задание

Внешняя спецификация программы:

Исходные данные:

- Исходная строка латинских символов.
- Выбор операции через консольное меню.

Выходные данные:

- Изображение дерева.
- Изображение дерева с повторяющимися элементами.
- Результаты операция удаления, поиска элементов.
- Сравнение времени выполнения и объема памяти операций удаления, поиска элементов.

Ограничения:

- Максимальный размер входной строки: 10 000 латинских символов.

Способ обращения: консольный ввод/вывод.

Команда запуска программы: ./app.exe. Дальнейшее взаимодействие осуществляется с помощью меню.

Аварийные ситуации:

- “ERROR_MEM” – Ошибка выделения памяти.
- “ERROR_IO” – Ошибка ввода.
- “ERROR_DELETING” – Попытка удалить элемент из пустого дерева.

Возможные действия, реализуемой программой:

0. Выход.
1. Добавить строку в дерево.
2. Удалить элемент из дерева.
3. Вывести дерево.
4. Искать элемент в дереве.
5. Вывести повторяющиеся элементы.
6. Удалить повторяющиеся элементы.
7. Очистить дерево.
8. Вывести три вида обхода дерева.
9. Сравнение производительности.

3. Внутренние структуры данных

В рамках выполнения данной лабораторной работы мной была реализована следующая структура данных:

Структура узла дерева

<pre>typedef struct node_t { char data; int count; struct node_t *left, *right; } node_t;</pre>

- **char** – хранящийся символ.
- **count** – количество вхождений символа.
- **left** – указатель на левого потомка.
- **right** – указатель на правого потомка.

4. Описание Алгоритмов

Основной алгоритм программы:

1. Вывести меню;
2. Считать выбор пользователя и проверить его корректность;
3. Выполнить соответствующее действие;
4. Либо снова вывести меню с ожиданием выбора, либо завершить программу с сообщением об ошибке.

Алгоритм вставки элемента в двоичное дерево поиска (insert):

1. Если корень дерева (root) равен NULL:
 - 1.1. Создать новый узел с данными data.
 - 1.2. Вернуть указатель на новый узел.
2. Иначе:
 - 2.1. Если data меньше значения в корне (root->data):
 - 2.1.1. Рекурсивно вставить data в левое поддерево.
 - 2.1.2. Присвоить результат в root->left.
 - 2.2. Иначе, если data больше значения в корне:
 - 2.2.1. Рекурсивно вставить data в правое поддерево.
 - 2.2.2. Присвоить результат в root->right.
 - 2.3. Иначе (значения равны):
 - 2.3.1. Увеличить счётчик повторений root->count на 1.
3. Вернуть указатель на корень (root).

Алгоритм поиска элемента в дереве (search):

1. Если корень root равен NULL или значение в корне совпадает с искомым key:
 - 1.1. Вернуть указатель на root (это либо найденный узел, либо NULL).
2. Иначе:
 - 2.1. Если key меньше значения в корне:
 - 2.1.1. Рекурсивно выполнить поиск в левом поддереве.
 - 2.2. Иначе:
 - 2.2.1. Рекурсивно выполнить поиск в правом поддереве.
3. Вернуть результат рекурсивного вызова.

Алгоритм удаления узла по ключу (delete_node):

1. Если корень root равен NULL:
 - 1.1. Вернуть NULL (элемент не найден).
2. Иначе:

- 2.1. Если key меньше значения в корне:
 - 2.1.1. Рекурсивно удалить key из левого поддерева.
 - 2.1.2. Присвоить результат в root->left.
- 2.2. Иначе, если key больше значения в корне:
 - 2.2.1. Рекурсивно удалить key из правого поддерева.
 - 2.2.2. Присвоить результат в root->right.
- 2.3. Иначе (найден удаляемый узел):
 - 2.3.1. Если у узла нет левого поддерева:
 - 2.3.1.1. Сохранить правое поддерево во временный указатель temp.
 - 2.3.1.2. Освободить память узла.
 - 2.3.1.3. Вернуть temp.
 - 2.3.2. Иначе, если у узла нет правого поддерева:
 - 2.3.2.1. Сохранить левое поддерево в temp.
 - 2.3.2.2. Освободить память узла.
 - 2.3.2.3. Вернуть temp.
 - 2.3.3. Иначе (у узла есть оба поддерева):
 - 2.3.3.1. Найти узел с минимальным значением в правом поддереве (find_min(root->right)).
 - 2.3.3.2. Скопировать данные (data и count) из этого узла в текущий корень.
 - 2.3.3.3. Рекурсивно удалить дубликат минимального узла из правого поддерева.
- 3. Вернуть указатель на обновлённый корень.

Алгоритм удаления всех дублирующихся узлов (delete_duplicates):

- 1. Если корень root равен NULL:
 - 1.1. Вернуть NULL.
- 2. Рекурсивно обработать левое и правое поддерева:
 - 2.1. Вызвать delete_duplicates(root->left), присвоить результат root->left.
 - 2.2. Вызвать delete_duplicates(root->right), присвоить результат root->right.
- 3. Если счётчик текущего узла больше 1 (root->count > 1):
 - 3.1. Удалить текущий узел по правилам, аналогичным delete_node:
 - 3.1.1. Если нет левого поддерева — вернуть правое.
 - 3.1.2. Если нет правого поддерева — вернуть левое.
 - 3.1.3. Если есть оба поддерева:
 - 3.1.3.1. Найти минимальный узел в правом поддереве.
 - 3.1.3.2. Скопировать его данные в текущий узел.
 - 3.1.3.3. Установить count = 1 (чтобы не удалить повторно).
 - 3.1.3.4. Рекурсивно удалить исходный минимальный узел из правого поддерева.

4. Вернуть указатель на корень (возможно, изменённый после удаления).

Алгоритм сравнения производительности операций:

1. Установить количество запусков.
2. Инициализировать переменные.
3. Замерить действие на строке.
4. Замерить действие на случайном дереве.
5. Замерить действие на вырожденным дереве.
6. Вывести сравнительную таблицу:
 - 6.1.Время работы (мкс).
 - 6.2.Используемая память (байт).

Реализуемые функции:

- // Функция очистки буфера ввода
- 1. void clear_input_buffer(void);

- // Функция проверки, состоит ли строка только из латинских букв
- 2. int is_string(char *string);

- // Функция вывода сообщения об ошибке по коду
- 3. void print_error(int rc);

- // Функция вывода главного меню программы
- 4. void print_menu(void);

- // Функция создания нового узла дерева с заданным символом
- 5. node_t *create_node(char data);

- // Функция вставки символа в двоичное дерево поиска с подсчётом дубликатов
- 6. node_t *insert(node_t *root, char data);

- // Функция поиска узла с заданным символом в дереве
- 7. node_t *search(node_t *root, char key);

- // Функция инфиксного (лево-корень-право) обхода дерева с отступами по глубине
- 8. void in_order(node_t *node, int depth);

- // Функция префиксного (корень-лево-право) обхода дерева с отступами по глубине
- 9. void pre_order(node_t *node, int depth);

- // Функция постфиксного (лево-право-корень) обхода дерева с отступами по глубине
- 10. void post_order(node_t *node, int depth);

```
    // Функция поиска узла с минимальным значением в поддереве  
11.node_t *find_min(node_t *root);
```

```
    // Функция удаления узла с заданным символом из дерева  
12.node_t *delete_node(node_t *root, char key);
```

```
    // Функция удаления всех узлов, символы в которых встречаются более  
    // одного раза  
13.node_t *delete_duplicates(node_t *root);
```

```
    // Функция освобождения всей памяти, занятой деревом  
14.void free_tree(node_t *root);
```

```
    // Функция вычисления высоты дерева  
15.int tree_height(const node_t *root);
```

```
    // Функция вычисления объёма памяти, занимаемой деревом (в байтах)  
16.size_t tree_memory_usage(const node_t *root);
```

```
    // Функция рекурсивной генерации содержимого DOT-файла с возможной  
    // подсветкой дубликатов  
17.void generate_dot_highlight(node_t *root, FILE *file, int highlight_duplicates);
```

```
    // Функция создания DOT-файла по дереву с опцией подсветки  
    // повторяющихся символов  
18.int create_dot_file(node_t *root, const char *filename, int highlight_duplicates);
```

```
    // Функция открытия изображения дерева с помощью системной команды  
19.void show_image(const char *image_path);
```

```
    // Функция удаления всех дубликатов символов из строки
```

20.void delete_str_duplicates(char *str);

// Функция линейного поиска символа в строке
21.int search_in_str(const char *str, char key);

// Функция генерации упорядоченной строки из циклически повторяющихся
букв латинского алфавита
22.void create_ordered_str(char *str, int size);

// Функция генерации случайной строки из латинских букв
23.void create_random_str(char *str, int size);

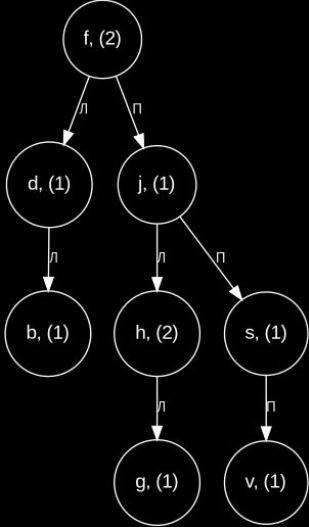
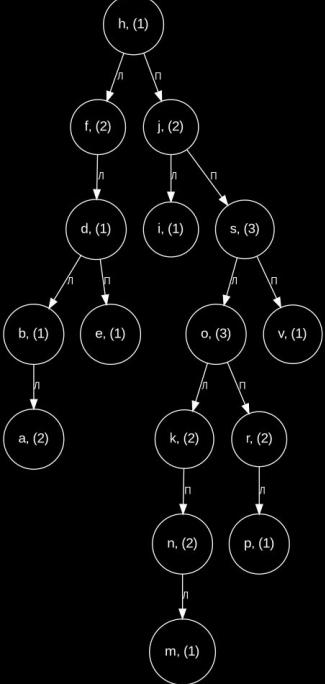
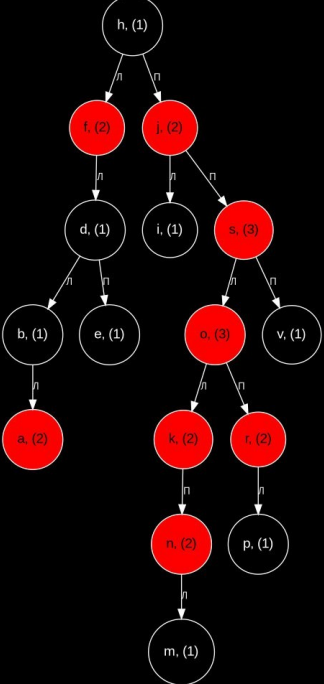
// Функция построения двоичного дерева поиска из заданной строки
24.node_t *create_tree_from_string(const char *str);

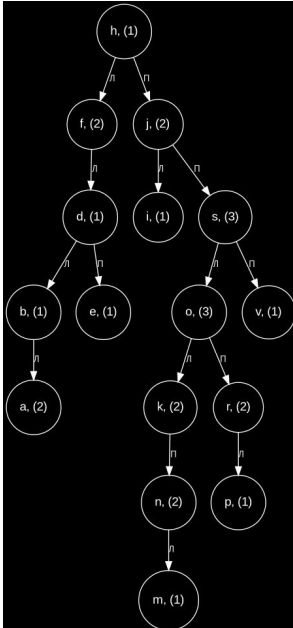
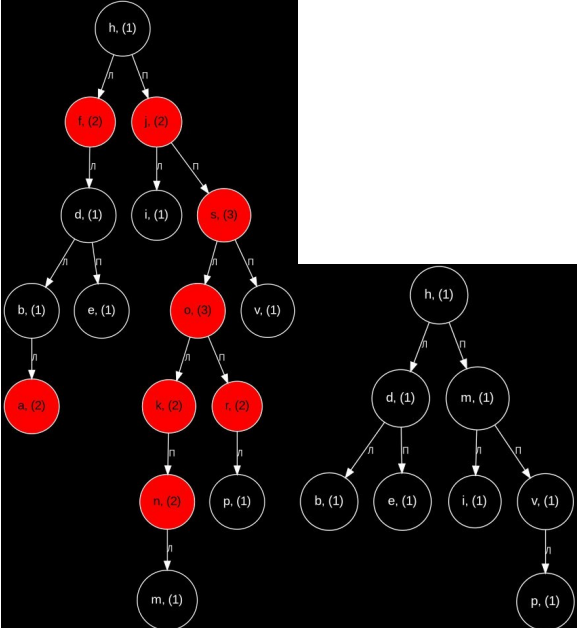
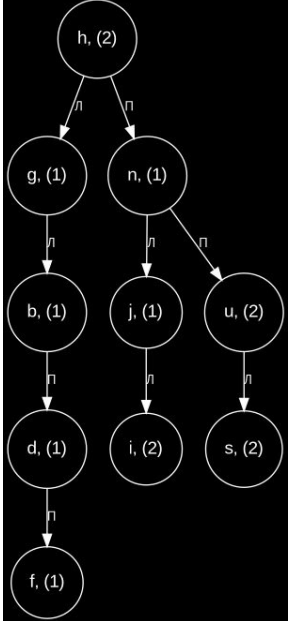
// Функция сравнения времени удаления дубликатов из дерева и из строки
25.void compare_delete(void);

// Функция сравнения времени поиска символа в дереве и в строке
26.void compare_find(void);

// Функция сравнения объёма памяти, занимаемого строкой и деревом при
одинаковом количестве символов
27.void compare_memory(void);

5. Тесты

## Позитивные тесты:	
Действие	Результат
Вывод пустого дерева	“Дерево пустое”
Вывод дерева из одного узла	
Вывод дерева из нескольких узлов: Строка: “fjdbsfhvgh”	
Вывод дубликатов	 

Удаление дубликатов				
Вывод видов обхода: Строка: “hgbnjudsfhuisi”		Инфиксны й: b (1) d (1) f (1) g (1) h (2) i (2) j (1) n (1) s (2) u (2)	Префиксны й: h (2) g (1) b (1) d (1) f (1) n (1) j (1) i (2) u (2) s (2)	Постфиксны й: f (1) d (1) b (1) g (1) i (2) j (1) s (2) u (2) n (1) h (2)
Удаление существующего и несуществующего элементов из дерева				
Поиск существующего и несуществующего элементов в дереве				

Негативные тесты:

1. Удаление из пустого дерева.
2. Ввод невалидного значения при выборе меню.
3. Ввод не латинского символа при вводе строки.

5. Оценка эффективности

Количество запусков – 200 раз.

Сравнение алгоритма удаления дубликатов

Кол-во символов	Высота случ. д.	Высота вырожд. д.	Время удаления случ. д., мкс.	Время удаления вырожд. д., мкс.	Время удаления строки, мкс.
100	9	26	0.695	0.455	5.055
200	9	26	1.155	0.845	17.065
500	10	26	0.685	0.440	85.060
1000	8	26	0.645	0.420	255.865
2000	10	26	0.715	0.460	931.045
5000	9	26	0.700	0.630	5499.460
10000	7	26	0.630	0.580	21391.785

Сравнение алгоритма поиска элемента

Кол-во символов	Высота случ. д.	Высота вырожд. д.	Время поиска случ. д., мкс.	Время поиска вырожд. д., мкс.	Время поиска строки, мкс.
100	8	26	0.190	0.315	0.215
200	8	26	0.240	0.330	0.265
500	9	26	0.225	0.275	0.235
1000	8	26	0.260	0.345	0.095
2000	8	26	0.200	0.355	0.340
5000	8	26	0.235	0.280	0.240
10000	12	26	0.240	0.305	0.310

Сравнение занимаемой памяти

Кол-во элементов	Строка, байт	Дерево, байт
100	100	2400
200	200	4800
500	500	12000
1000	1000	24000
2000	2000	48000
5000	5000	120000
10000	10000	240000

Наблюдения результатов по эффективности:

1. Алгоритм удаления дубликатов: дерево значительно превосходит строку по скорости.
 - 1.1. При всех размерах данных (от 100 до 10 000 символов) время удаления из дерева на порядки меньше, чем из строки.
 - 1.2. Например, при $N=10\,000$: Время удаления из строки — 21 391 мкс, время удаления из случайного дерева — 630 мкс, из вырожденного — 580 мкс.
 - 1.3. Высота дерева не оказывает значительного влияния на время удаления дубликатов.
2. Поиск элемента: дерево и строка показывают сопоставимую производительность.
 - 2.1. При малых N (до 1000) время поиска в строке даже немного лучше, чем в дереве.
 - 2.2. При больших N (5000–10 000) дерево начинает выигрывать, но преимущество минимально — разница в пределах 10–20%.
 - 2.3. Это объясняется тем, что линейный поиск в строке имеет $O(N)$, а поиск в BST — $O(\log N)$ в среднем, однако при малых N накладные расходы на рекурсию и переходы по указателям в дереве сводят это преимущество на нет.
 - 2.4. Высота дерева влияет на время поиска, но не критично.
3. Память: дерево требует значительно больше памяти, чем строка.
4. Соотношение памяти «дерево/строка» постоянно равно 24/1 для всех размеров.
5. Таким образом, дерево платит за скорость огромным перерасходом памяти.

6. Выводы

Итак, можно сделать следующий вывод, что операции над деревом (особенно удаление дубликатов) выполняются значительно быстрее, чем эквивалентные операции над строкой, благодаря логарифмической сложности. Однако эта выгода достигается за счёт существенного перерасхода памяти — дерево занимает в ~24 раза больше байт, чем строка.

Таким образом, двоичные деревья целесообразно использовать в задачах, где важна высокая скорость динамической обработки данных (частые вставки, удаления, поиск), но нецелесообразны при работе с статичными или компактными наборами данных, где критична экономия памяти.

7. Ответы на контрольные вопросы

1. Что такое дерево?

Дерево — это абстрактная структура данных, представляющая собой иерархическую модель, в которой элементы связаны друг с другом через отношения "родитель-потомок". Оно состоит из узлов (вершин), соединённых рёбрами, где:

- у каждого узла, кроме корневого, есть один родитель;
- узел без потомков называется листом;
- у дерева есть только один корень;

Выделение памяти под представление деревьев:

Память под дерево обычно выделяется динамически, поскольку количество узлов заранее неизвестно. Узел дерева может быть представлен структурой или объектом в памяти.

2. Какие бывают типы деревьев?

1. Обычное дерево (general tree):
 - узел может иметь любое количество потомков.
 - применяется для представления файловых систем, иерархий.
2. Бинарное дерево (binary tree):
 - каждый узел имеет не более двух потомков: левого и правого.
3. Дерево двоичного поиска (binary search tree, BST):
 - упорядоченное бинарное дерево, где для каждого узла выполняется условие: левые потомки содержат меньшие значения, а правые потомки содержат большие значения.
4. Сбалансированное дерево (balanced tree):
 - обеспечивается равномерное распределение узлов для минимизации глубины дерева
5. Декартово дерево (Cartesian tree):
 - комбинация бинарного дерева поиска и кучи.
6. Префиксное дерево (Trie):
 - используется для хранения строк и поиска по префиксу

3. Какие стандартные операции возможны над деревьями?

1. Обход дерева
 - Прямой (pre-order): Посещение узла, затем обход левого и правого поддерева.
 - Симметричный (in-order): Обход левого поддерева, посещение узла, затем

- правого поддерева.
- Обратный (post-order): Обход левого и правого поддерева, затем посещение
- узла.
- По уровням (level-order): Обход всех узлов на одном уровне перед переходом на следующий.

2. Поиск узла:

- Поиск узла с определённым значением (особенно эффективен в дереве двоичного поиска)

3. Вставка узла:

- Добавление нового узла в дерево с сохранением его свойств.

4. Удаление узла:

- Удаление узла с корректировкой связей между оставшимися элементами.

4. Что такое дерево двоичного поиска?

Дерево двоичного поиска - это специальный тип бинарного дерева, в котором выполняется условие: значения в левом поддерева узла меньше значения самого узла, а значения в правом поддерева узла больше значения самого узла.